

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Linq;
using System.Runtime.InteropServices;
using System.Text;
using System.Threading.Tasks;
namespace GoProxyTool
{

    public class IEProxySetting
    {
        public static bool UnsetProxy()
        {
            return SetProxy(null, null);
        }
        public static bool SetProxy(string strProxy)
        {
            return SetProxy(strProxy, null);
        }
        public static bool SetProxy(string strProxy, string exceptions)
        {
            InternetPerConnOptionList list = new InternetPerConnOptionList();
            int optionCount = string.IsNullOrEmpty(strProxy) ? 1 :
(string.IsNullOrEmpty(exceptions) ? 2 : 3);
            InternetConnectionOption[] options = new
InternetConnectionOption[optionCount];

            options[0].m_Option = PerConnOption.INTERNET_PER_CONN_FLAGS;
            options[0].m_Value.m_Int = (int)((optionCount < 2) ?
PerConnFlags.PROXY_TYPE_DIRECT : (PerConnFlags.PROXY_TYPE_DIRECT |
PerConnFlags.PROXY_TYPE_PROXY));

            if (optionCount > 1)
            {
                options[1].m_Option =
PerConnOption.INTERNET_PER_CONN_PROXY_SERVER;
                options[1].m_Value.m_StringPtr =
Marshal.StringToHGlobalAuto(strProxy);
                except for these addresses ...
                if (optionCount > 2)
                {
                    options[2].m_Option =
PerConnOption.INTERNET_PER_CONN_PROXY_BYPASS;
                    options[2].m_Value.m_StringPtr =
Marshal.StringToHGlobalAuto(exceptions);
                }
            }
            list.dwSize = Marshal.SizeOf(list);
            list.szConnection = IntPtr.Zero;

```

```

        list.dwOptionCount = options.Length;
        list.dwOptionError = 0;
        int optSize = Marshal.SizeOf(typeof(InternetConnectionOption));

        IntPtr optionsPtr = Marshal.AllocCoTaskMem(optSize * options.Length);

        for (int i = 0; i < options.Length; ++i)
        {
            IntPtr opt = new IntPtr(optionsPtr.ToInt32() + (i * optSize));
            Marshal.StructureToPtr(options[i], opt, false);
        }
        list.options = optionsPtr;

        IntPtr ipcoListPtr = Marshal.AllocCoTaskMem((Int32)list.dwSize);
        Marshal.StructureToPtr(list, ipcoListPtr, false);

        int returnvalue = NativeMethods.InternetSetOption(IntPtr.Zero,
            InternetOption.INTERNET_OPTION_PER_CONNECTION_OPTION,
            ipcoListPtr, list.dwSize) ? -1 : 0;
        if (returnvalue == 0)
        {
            returnvalue = Marshal.GetLastWin32Error();
        }
        FREE the data ASAP
        Marshal.FreeCoTaskMem(optionsPtr);
        Marshal.FreeCoTaskMem(ipcoListPtr);
        if (returnvalue > 0)
        {
            throw new Win32Exception(Marshal.GetLastWin32Error());
        }
        return (returnvalue < 0);
    }
}

#region WinInet structures
[StructLayout(LayoutKind.Sequential, CharSet = CharSet.Auto)]
public struct InternetPerConnOptionList
{
    public int dwSize;
    public IntPtr szConnection;
    public int dwOptionCount;
    public int dwOptionError;
    [MarshalAs(UnmanagedType.))
    public IntPtr options;
};

[StructLayout(LayoutKind.Sequential, CharSet = CharSet.Auto)]
public struct InternetConnectionOption
{
    static readonly int Size;
    public PerConnOption m_Option;
    public InternetConnectionOptionValue m_Value;

```

```

        static InternetConnectionOption()
        {
            InternetConnectionOption.Size =
Marshal.SizeOf(typeof(InternetConnectionOption));
        }

        [StructLayout(LayoutKind.Explicit)]
        public struct InternetConnectionOptionValue
        {

            [FieldOffset(0)]
            public System.Runtime.InteropServices.ComTypes.FILETIME m_FileTime;
            [FieldOffset(0)]
            public int m_Int;
            [FieldOffset(0)]
            public IntPtr m_StringPtr;
        }
    }
    #endregion
    #region WinInet enums

    public enum InternetOption : uint
    {
        INTERNET_OPTION_PER_CONNECTION_OPTION = 75
    }

    public enum PerConnOption
    {
        INTERNET_PER_CONN_FLAGS = 1,
        INTERNET_PER_CONN_PROXY_SERVER = 2,
        INTERNET_PER_CONN_PROXY_BYPASS = 3,
        INTERNET_PER_CONN_AUTOCONFIG_URL = 4,
    }

    public enum PerConnFlags
    {
        PROXY_TYPE_DIRECT = 0x00000001,
        PROXY_TYPE_PROXY = 0x00000002,
        PROXY_TYPE_AUTO_PROXY_URL = 0x00000004,
        PROXY_TYPE_AUTO_DETECT = 0x00000008
    }
    #endregion
    internal static class NativeMethods
    {
        [DllImport("WinInet.dll", SetLastError = true, CharSet = CharSet.Auto)]
        [return: MarshalAs(UnmanagedType.Bool)]
        public static extern bool InternetSetOption(IntPtr hInternet,
InternetOption dwOption, IntPtr lpBuffer, int dwBufferLength);
    }
}
using System;

```

```

using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
namespace IEProxyManagment.Models
{
    [Serializable]
    public class Server
    {
        #region ParseLegacyURL
        public static readonly Regex
            UrlFinder = new Regex(@"ss:
            DetailsParser
            =
            new
            Regex(@"^((?<method>. +?): (?<password>.* )@(?<hostname>. +?): (?<port>\d+?))$",
            RegexOptions.IgnoreCase);
        #endregion ParseLegacyURL
        private const int DefaultServerTimeoutSec = 5;
        public const int MaxServerTimeoutSec = 20;
        public string server;
        public int server_port;
        public string server_type;
        public string server_model;
        public string password;
        public string method;
        public string plugin;
        public string plugin_opts;
        public string plugin_args;
        public string remarks;
        public int timeout;
        public override int GetHashCode()
        {
            return server.GetHashCode() ^ server_port;
        }
        public override bool Equals(object obj)
        {
            Server o2 = (Server)obj;
            return server == o2.server && server_port == o2.server_port;
        }
        public string FriendlyName()
        {
            if (server.IsNullOrEmpty())
            {
                return I18N.GetString("New server");
            }
            string serverStr = $"{FormatHostName(server)}:{server_port}";
            return remarks.IsNullOrEmpty()
                ? serverStr
                : $"{remarks} ({serverStr})";
        }
    }
}

```

```

public string FormatHostName(string hostName)
{
    switch (Uri.CheckHostName(hostName))
    {
        case UriHostNameType.Ipv6:
            return $"[{hostName}]";
        default:
            IPv4 or domain name
            return hostName;
    }
}

public Server()
{
    server = "";
    server_port = 8090;
    server_type = "http";
    server_model = "ws";
    method = "aes-256-cfb";
    password = "";
}

private static Server ParseLegacyURL(string ssURL)
{
    var match = UrlFinder.Match(ssURL);
    if (!match.Success)
    {
        return null;
    }
    Server server = new Server();
    var base64 = match.Groups["base64"].Value.TrimEnd('/');
    var tag = match.Groups["tag"].Value;
    if (!tag.IsNullOrEmpty())
    {
        server.remarks = HttpUtility.UrlDecode(tag, Encoding.UTF8);
    }
    Match details = null;
    try
    {
        details =
        DetailsParser.Match(Encoding.UTF8.GetString(Convert.FromBase64String(
            base64.PadRight(base64.Length + (4 - base64.Length % 4) % 4,
            '='))));
    }
    catch (FormatException)
    {
        return null;
    }
    if (!details.Success)
    {
        return null;
    }
    server.method = details.Groups["method"].Value;
    server.password = details.Groups["password"].Value;
    server.server = details.Groups["hostname"].Value;
}

```

```

        server.server_port = int.Parse(details.Groups["port"].Value);
        return server;
    }
    public static List<Server> GetServers(string ssURL)
    {
        var serverUrls = ssURL.Split('\'\'', '\n', '\r');
        List<Server> servers = new List<Server>();
        foreach (string serverUrl in serverUrls)
        {
            string _serverUrl = serverUrl.Trim();
            if (!_serverUrl.StartsWith("ss:",
StringComparison.InvariantCultureIgnoreCase))
            {
                continue;
            }
            Server legacyServer = ParseLegacyURL(serverUrl);
            if (legacyServer != null)    legacy
            {
                servers.Add(legacyServer);
            }
            else    SIP002
            {
                Uri parsedUrl;
                try
                {
                    parsedUrl = new Uri(serverUrl);
                }
                catch (UriFormatException)
                {
                    continue;
                }
                Server server = new Server
                {
                    remarks = parsedUrl.GetComponents(UriComponents.Fragment,
UriFormat.Unescaped),
                    server = parsedUrl.IdnHost,
                    server_port = parsedUrl.Port,
                };
                parse base64 UserInfo
                string rawUserInfo =
parsedUrl.GetComponents(UriComponents.UserInfo, UriFormat.Unescaped);
                string base64 = rawUserInfo.Replace('-', '+').Replace('_',
\'/');    Web-safe base64 to normal base64
                string userInfo = "";
                try
                {
                    userInfo =
Encoding.UTF8.GetString(Convert.FromBase64String(
base64.PadRight(base64.Length + (4 - base64.Length % 4) %
4, \'=\')));

```

```

        }
        catch (FormatException)
        {
            continue;
        }
        string[] userInfoParts = userInfo.Split(new char[] { ':' }, 2);
        if (userInfoParts.Length != 2)
        {
            continue;
        }
        server.method = userInfoParts[0];
        server.password = userInfoParts[1];
        NameValueCollection queryParameters =
HttpUtility.ParseQueryString(parsedUrl.Query);
        string[] pluginParts = (queryParameters["plugin"] ??
        "").Split(new[] { ';' }, 2);
        if (pluginParts.Length > 0)
        {
            server.plugin = pluginParts[0] ?? "";
        }
        if (pluginParts.Length > 1)
        {
            server.plugin_opts = pluginParts[1] ?? "";
        }
        servers.Add(server);
    }
}
return servers;
}
public string Identifier()
{
    return server + ':' + server_port;
}
}
}
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Management;
using System.Text;
using System.Threading.Tasks;
namespace GoProxyTool
{
    public static class ProcessEx
    {
        public static void KillProcessTree(this Process parent)
        {
            var searcher = new ManagementObjectSearcher("Select * From

```

```

Win32_Process Where ParentProcessID=" + parent.Id);
    var moc = searcher.Get();
    foreach (ManagementObject mo in moc)
    {
        Process childProcess =
Process.GetProcessById(Convert.ToInt32(mo["ProcessID"]));
        childProcess.KillProcessTree();
    }
    Console.WriteLine(string.Format("kill process by id {0}!",
parent.Id));
    if (parent.Id != Process.GetCurrentProcess().Id)
        parent.Kill();
}
public static void KillProcessTreeById(int pid)
{
    try
    {
        var process = Process.GetProcessById(pid);
        try
        {
            process.KillProcessTree();
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
    }
    catch
    {
        Console.WriteLine("failed to get the process!");
        var searcher = new ManagementObjectSearcher("Select * From
Win32_Process Where ParentProcessID=" + pid);
        var moc = searcher.Get();
        foreach (ManagementObject mo in moc)
        {
            Process childProcess =
Process.GetProcessById(Convert.ToInt32(mo["ProcessID"]));
            childProcess.KillProcessTree();
        }
    }
}
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace GoProxyTool
{

```



```
static class Program
{
    static void Main()
    {
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        Application.Run(new ProxySettingWin());
    }
}

using System.Reflection;
using System.Runtime.CompilerServices;
using System.Runtime.InteropServices;
namespace GoProxyTool.Properties {
    using System;
    internal class Resources {

        private static global::System.Resources.ResourceManager resourceMan;

        private static global::System.Globalization.CultureInfo resourceCulture;

        [global::System.Diagnostics.CodeAnalysis.SuppressMessageAttribute("Microsoft.Performance", "CA1811:AvoidUncalledPrivateCode")]
        internal Resources() {
        }

        internal static global::System.Resources.ResourceManager ResourceManager
        {
            get {
                if (object.ReferenceEquals(resourceMan, null)) {
                    global::System.Resources.ResourceManager temp = new
global::System.Resources.ResourceManager("GoProxyTool.Properties.Resources",
typeof(Resources).Assembly);
                    resourceMan = temp;
                }
                return resourceMan;
            }
        }

        internal static global::System.Globalization.CultureInfo Culture {
            get {
                return resourceCulture;
            }
            set {
                resourceCulture = value;
            }
        }
    }
}
```

```

    }
}
namespace GoProxyTool.Properties {

    internal sealed partial class Settings :
global::System.Configuration.ApplicationSettingsBase {

        private static Settings defaultInstance =
((Settings) (global::System.Configuration.ApplicationSettingsBase.Synchronized(n
ew Settings())));

        public static Settings Default {
            get {
                return defaultInstance;
            }
        }
    }
}
using GoProxyTool.Properties;
using Microsoft.Win32;
using Newtonsoft.Json;
using Newtonsoft.Json.Converters;
using Newtonsoft.Json.Linq;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Diagnostics;
using System.Drawing;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace GoProxyTool
{
    public partial class ProxySetingWin : Form
    {
        public Process process = null;
        public ProxySetingWin()
        {
            InitializeComponent();
        }

        private void btnSetProxy_Click(object sender, EventArgs e)
        {
            //string proxyStr = combProxyList.Text.Trim();
            string proxyStr = "http://127.0.0.1:" + textBox4.Text;
            IEProxySetting.SetProxy(proxyStr, null);
        }
    }
}

```

```

var currentProxy = GetProxyServer();
if (currentProxy == proxyStr && GetProxyStatus())
{
    lblInfo.Text = ": " + proxyStr + "! ";
    lblInfo.ForeColor = Color.Green;
}
else
{
    if (!GetProxyStatus())
    {
        lblInfo.Text = ": " + proxyStr + "! ";
    }
    else
    {
        lblInfo.Text = ": " + proxyStr + ", " + currentProxy + ", !
";
    }
    lblInfo.ForeColor = Color.Red;
}
ShowProxyInfo();
}
private void ProxySetingWin_Load(object sender, EventArgs e)
{
    InitConfig();
    comboBox2.SelectedIndex = 0;
    string proxyPath = workPath + "\\\" + appName;
    //MessageBox.Show(proxyPath);
    if (!File.Exists(proxyPath))
    {
        MessageBox.Show(".!");
        System.Environment.Exit(0);
    }
    else
    {
        ShowProxyInfo();
        InitProxyData();
        button2.Enabled = false;
    }
}
private string GetProxyServer()
{
    RegistryKey regKey = Registry.CurrentUser;
    string SubKeyPath = "Software\\Microsoft\\Windows\\CurrentVersion\\Internet Settings";
    RegistryKey optionKey = regKey.OpenSubKey(SubKeyPath,
true);
    string actualProxy = optionKey.GetValue("ProxyServer").ToString();
    regKey.Close();

```

```

        return actualProxy;
    }
    private bool GetProxyStatus()
    {
        RegistryKey regKey = Registry.CurrentUser;
        string SubKeyPath =
"Software\\Microsoft\\Windows\\CurrentVersion\\Internet Settings";
        RegistryKey optionKey = regKey.OpenSubKey(SubKeyPath,
true);
        int actualProxyStatus =
Convert.ToInt32(optionKey.GetValue("ProxyEnable"));
        regKey.Close();
        return actualProxyStatus == 1 ? true : false;
    }
    private void ShowProxyInfo()
    {
        if (!GetProxyStatus())
        {
            lblInitInfo.Text = ": ";
        }
        else
        {
            lblInitInfo.Text = ": " + GetProxyServer();
        }
    }
    private void InitProxyData()
    {
        List<string> proxyList = new List<string>{
            "http://127.0.0.1:8090", "http://proxy.compaq.com:8080"
        };
        combProxyList.DataSource = proxyList;
        combProxyList.SelectedIndex = 0;
    }
    private void InitParentMethodData()
    {
        List<string> ssMethodList = new List<string>{
            "",
            "aes-128-cfb",
            "aes-128-ctr",
            "aes-128-gcm",
            "aes-192-cfb",
            "aes-192-ctr",
            "aes-192-gcm",
            "aes-256-cfb",
            "aes-256-ctr",
            "aes-256-gcm",
            "bf-cfb",
            "cast5-cfb",

```

```
        "chacha20" ,
        "chacha20-ietf" ,
        "chacha20-ietf-poly1305" ,
        "des-cfb" ,
        "rc4-md5" ,
        "rc4-md5-6" ,
        "salsa20" ,
        "xchacha20"
    };
    List<string> httpMethodList = new List<string>{
        "",
        "aes-128-cfb",
        "aes-128-ctr",
        "aes-192-cfb",
        "aes-192-ctr",
        "aes-256-cfb",
        "aes-256-ctr",
        "bf-cfb",
        "cast5-cfb",
        "chacha20",
        "chacha20-ietf",
        "des-cfb",
        "rc4-md5",
        "rc4-md5-6"
    };
    List<string> socksMethodList = new List<string>{
        "",
        "aes-128-cfb",
        "aes-128-ctr",
        "aes-192-cfb",
        "aes-192-ctr",
        "aes-256-cfb",
        "aes-256-ctr",
        "bf-cfb",
        "cast5-cfb",
        "chacha20",
        "chacha20-ietf",
        "des-cfb",
        "rc4-md5",
        "rc4-md5-6"
    };
    List<string> httpTypeList = new List<string>{
        "ws",
        "wss"
    };
    List<string> ssTypeList = new List<string>{
        "tcp"
    };
    List<string> socksTypeList = new List<string>{
        "ws",
```

```

        "wss"
    };
    if (comboBox2.Text == "http")
    {
        comboBox4.DataSource = httpMethodList;
        comboBox4.SelectedIndex = 0;
        comboBox1.DataSource = httpTypeList;
        comboBox1.SelectedIndex = 0;
    }
    else if (comboBox2.Text == "ss")
    {
        comboBox4.DataSource = ssMethodList;
        comboBox4.SelectedIndex = 0;
        comboBox1.DataSource = ssTypeList;
        comboBox1.SelectedIndex = 0;
    }
    else if (comboBox2.Text == "socks")
    {
        comboBox4.DataSource = socksMethodList;
        comboBox4.SelectedIndex = 0;
        comboBox1.DataSource = socksTypeList;
        comboBox1.SelectedIndex = 0;
    }
}

private void btnDisableProxy_Click(object sender, EventArgs e)
{
    if (IEProxySetting.UnsetProxy())
    {
        MessageBox.Show("! ");
    }
    else
    {
        MessageBox.Show("! ");
    }
    if (!GetProxyStatus())
    {
        lblInfo.Text = "! ";
        lblInfo.ForeColor = Color.Green;
    }
    else
    {
        lblInfo.Text = ", " + GetProxyServer();
        lblInfo.ForeColor = Color.Red;
    }
    ShowProxyInfo();
}

private void Button3_Click(object sender, EventArgs e)
{
    OpenFileDialog dialog = new OpenFileDialog();
    System.Windows.Forms.FolderBrowserDialog dialog = new

```

```
System.Windows.Forms.FolderBrowserDialog();
    dialog.Multiselect = true;
    dialog.Title = "";
    dialog.Filter = "(*.exe)|*.exe";
    if (dialog.ShowDialog() == System.Windows.Forms.DialogResult.OK)
    {
        textBox2.Text = dialog.FileName;
        textBox4.Text = System.IO.Path.GetDirectoryName(dialog.FileName)
+ "\\Converter\\" + dialog.SafeFileName;
    }
}

string appName = "proxy.exe";
string workPath = System.IO.Directory.GetCurrentDirectory();
private void Button4_Click(object sender, EventArgs e)
{
    textBox3.Text = "";
    textBox6.Text = "8880";
    comboBox2.Text = "http";
    comboBox1.Text = e.server_mode;
    comboBox4.Text = e.method;
    textBox7.Text = "";
}

private string StrMake(string parameter, string str)
{
    if (parameter.Contains("="))
    {
        if (str == "")
        {
            return "";
        }
        else
        {
            return " " + parameter + "\"\" + str + "\"\"";
        }
    }
    else
    {
        if (str == "")
        {
            return "";
        }
        else
        {
            return " " + parameter + " \"\" + str + "\"\"";
        }
    }
}
```

```

private string StrMakeWithEqual(string parameter, string str)
{
    if (str == "")
    {
        return "";
    }
    else
    {
        return " " + parameter + "\"\" + str + "\"\"";
    }
}

private void Button1_Click(object sender, EventArgs e)
{
    string parent_ws_method = "--parent-ws-method=";
    string parent_ws_password = "--parent-ws-password=";
    string local_ws_method = "--local-ws-method=";
    string local_ws_password = "--local-ws-password=";
    string parent_ss_method = "--parent-ss-method=";
    string parent_ss_password = "--parent-ss-password=";
    string local_ss_method = "--ss-method=";
    string local_ss_password = "--ss-key=";
    string sps = "sps";
    switch (comboBox2.Text)
    {
        case "http":
            textBox1.Text = appName + " " + sps + StrMake("-t", "tcp") +
StrMake("-p", ":" + textBox4.Text) + StrMake("-T", comboBox1.Text) + StrMake("-P",
textBox3.Text + ":" + textBox6.Text) + StrMake(parent_ws_method, comboBox4.Text)
+ StrMake(parent_ws_password, textBox7.Text);
            break;
        case "ss":
            textBox1.Text = appName + " " + sps + StrMake("-t", "tcp") +
StrMake("-p", ":" + textBox4.Text) + StrMake("-T", comboBox1.Text) + StrMake("-P",
textBox3.Text + ":" + textBox6.Text) + StrMake(parent_ss_method, comboBox4.Text)
+ StrMake(parent_ss_password, textBox7.Text);
            break;
        case "socks":
            textBox1.Text = appName + " " + sps + StrMake("-t", "tcp") +
StrMake("-p", ":" + textBox4.Text) + StrMake("-T", comboBox1.Text) + StrMake("-P",
textBox3.Text + ":" + textBox6.Text) + StrMake(parent_ws_method, comboBox4.Text)
+ StrMake(parent_ws_password, textBox7.Text);
            break;
    }
    OutPutForm_Shown(textBox1.Text);
    executeCmd(textBox2.Text);
    button1.Enabled = false;
    button2.Enabled = true;
}

private void OutPutForm_Shown(string CMDcommand)

```



```

{
    Control.CheckForIllegalCrossThreadCalls = false;
    process = new Process();
    process.StartInfo.FileName = "cmd.exe";
    process.StartInfo.WorkingDirectory = ".";
    process.StartInfo.UseShellExecute = false;
    process.StartInfo.RedirectStandardInput = true;
    process.StartInfo.RedirectStandardOutput = true;
    process.StartInfo.RedirectStandardError = true;
    process.StartInfo.CreateNoWindow = true;
    Process.Start("cmd.exe");
    process.OutputDataReceived += new
DataReceivedEventHandler(OutputHandler);
    process.ErrorDataReceived += new
DataReceivedEventHandler(OutputHandler);
    process.Start();
    process.StandardInput.WriteLine(CMDcommand);
    process.StandardInput.WriteLine("exit");
    process.BeginOutputReadLine();
    process.BeginErrorReadLine();
    using (StreamWriter sw = new StreamWriter("output.log"))
    {
        sw.WriteLine(process.StandardOutput.ReadToEnd());
    }
}

private void OutputHandler(object sendingProcess, DataReceivedEventArgs
outLine)
{
    if (!String.IsNullOrEmpty(outLine.Data))
    {
        StringBuilder sb = new StringBuilder(this.textBox5.Text);
        this.textBox5.Text = sb.AppendLine(outLine.Data).ToString();
        this.textBox5.SelectionStart = this.textBox5.Text.Length;
        this.textBox5.ScrollToCaret();
    }
}

private void Button2_Click(object sender, EventArgs e)
{
    if (process != null)
    {
        ProcessEx.KillProcessTreeById(process.Id);
        process.Close();
    }
    button1.Enabled = true;
    button2.Enabled = false;
    textBox5.AppendText("。。。" + "\r\n");
}

private void ProxySetingWin_FormClosing(object sender,
FormClosingEventArgs e)
{

```

```
        if (process != null)
        {
            try
            {
                ProcessEx.KillProcessTreeById(process.Id);
                process.Close();
            }
            catch
            {
                ;
            }
        }
        System.Environment.Exit(0);
    }
    private void Button3_Click_1(object sender, EventArgs e)
    {
        if (button3.Text == "")
        {
            this.Size = new Size(903, 517);
            button3.Text = "";
        }
        else if (button3.Text == "")
        {
            this.Size = new Size(425, 517);
            button3.Text = "";
        }
    }
    private void ComboBox2_SelectedValueChanged(object sender, EventArgs e)
    {
        InitParentMethodData();
    }
    public class ConfigsItem
    {
        public string server { get; set; }

        public int server_port { get; set; }

        public string server_type { get; set; }

        public string server_mode { get; set; }

        public string method { get; set; }

        public string password { get; set; }
    }
    public class GoProxyConfigs
    {
        public int index { get; set; }
```

```

        public List<ConfigsItem> configs { get; set; }
    }
    public List<ConfigsItem> configs = new List<ConfigsItem>();
    public GoProxyCongfigs rb;
    public int mainIndex = 0;
    public int maxIndex = 0;
    public string jsonconfigpath = @"ui_config.json";
    private void InitConfig()
    {
        configs.Clear();
        listBox1.Items.Clear();

        string jsonfile = jsonconfigpath;
        if (!File.Exists(jsonfile))
        {
            File.Create(con_file_path);
            int index = 0;
            ConfigsItem configsItem = new ConfigsItem();
            configsItem.server = "your.domain or ip";
            configsItem.server_port = 8880;
            configsItem.server_type = "http";
            configsItem.server_mode = "ws";
            configsItem.method = "aes-256-cfb";
            configsItem.password = "abc";
            configs.Add(configsItem);
            Writejson(index, configs, jsonconfigpath);
            configs.Clear();
        }

        using (System.IO.StreamReader file =
System.IO.File.OpenText(jsonfile))
        {
            string json = file.ReadToEnd();
            rb = JsonConvert.DeserializeObject<GoProxyCongfigs>(json);
            mainIndex = rb.index;
            maxIndex = rb.configs.Count() - 1;
            int n = 0;
            foreach (ConfigsItem e in rb.configs)
            {
                if(n==mainIndex)
                {
                    textBox3.Text = e.server;//ip
                    textBox6.Text = e.server_port.ToString();
                    comboBox2.Text = e.server_type;
                    comboBox1.Text = e.server_mode;
                    comboBox4.Text = e.method;
                    textBox7.Text = e.password;

                }
                listBox1.Items.Add(e.server + ":" + e.server_port.ToString() +

```

```

"\n");

        MessageBox.Show(n.ToString());
        n++;
        configs.Add(e);
    }

}

Writejson(configs, @"config0000.json");
}

private void Button5_Click(object sender, EventArgs e)
{
    ConfigsItem configsItem = new ConfigsItem();
    configsItem.server = textBox3.Text;//ip
    configsItem.server_port = Convert.ToInt32(textBox6.Text);
    configsItem.server_type = comboBox2.Text;
    configsItem.server_mode = comboBox1.Text;
    configsItem.method = comboBox4.Text;
    configsItem.password = textBox7.Text;
    configs.Add(configsItem);
    int index = maxIndex + 1;
    Writejson(index, configs, jsonconfigpath);
    InitConfig();
}

public void Writejson(int index, List<ConfigsItem> configs, string path)
{
    Hashtable device = new Hashtable();
    device.Add("id", "001111");
    device.Add("name", "");
    device.Add("ip", "192.168.1.1");
    GoProxyCongfigs lot = new GoProxyCongfigs();
    lot.Name = "lotname";
    lot.Address = "wenzhou";
    lot.index = index;
    lot.configs = configs;
    lot.Doors = "4";

    string js1 = JsonConvert.SerializeObject(lot);
    string js2 = JsonConvert.SerializeObject(device);

    GoProxyCongfigs debc1 =
JsonConvert.DeserializeObject<GoProxyCongfigs>(js1);
    LotModel debc2 = JsonConvert.DeserializeObject<LotModel>(js2);
    string serverAppPath = Request.PhysicalApplicationPath.ToString();
    string serverAppPath = @"d:\";

    string con_file_path = @"" + serverAppPath + @"config.json";
    string con_file_path = path;

```

```
        if (!File.Exists(con_file_path))
        {
            File.Create(con_file_path);
            File.Create(con_file_path).Close();
        }

        using (StreamWriter sw = new StreamWriter(con_file_path))
        {
            JsonSerializer serializer = new JsonSerializer();
            serializer.Converters.Add(new JavaScriptDateTimeConverter());
            serializer.NullValueHandling = NullValueHandling.Ignore;

            JsonWriter writer = new JsonTextWriter(sw);

            serializer.Serialize(writer, lot);
            ser.Serialize(writer, ht);
            writer.Close();
            sw.Close();
        }
    }

    private void ListBox1_MouseDoubleClick(object sender, MouseEventArgs e)
    {
        mainIndex = listBox1.SelectedIndex;
        ConfigsItem cf = rb.configs[listBox1.SelectedIndex];
        textBox3.Text = cf.server;
        textBox6.Text = cf.server_port.ToString();
        comboBox2.Text = cf.server_type;
        comboBox1.Text = cf.server_mode;
        comboBox4.Text = cf.method;
        textBox7.Text = cf.password;
        int index = mainIndex;
        Color vColor = Color.Gainsboro;
        Graphics devcolor = listBox1.CreateGraphics();
        vColor = Color.Lime;
        devcolor.FillRectangle(new SolidBrush(vColor),
listBox1.GetItemRectangle(index));
        devcolor.DrawString(listBox1.Items[index].ToString(), listBox1.Font,
new SolidBrush(listBox1.ForeColor), listBox1.GetItemRectangle(index));

        Writejson(index, configs, jsonconfigpath);
        InitConfig();
    }

    private void(object sender, EventArgs e)
    {
        listBox1.Items.Remove(listBox1.SelectedItems[listBox1.SelectedIndex]);
        configs.RemoveAt(listBox1.SelectedIndex);
        int index = listBox1.SelectedIndex-1;
        Color vColor = Color.Gainsboro;
        Graphics devcolor = listBox1.CreateGraphics();
```

```

        vColor = Color.Lime;
        devcolor.FillRectangle(new SolidBrush(vColor),
listBox1.GetItemRectangle(index));
        devcolor.DrawString(listBox1.Items[index].ToString(), listBox1.Font,
new SolidBrush(listBox1.ForeColor), listBox1.GetItemRectangle(index));
        if (index <= 0)
        {
            index = 0;
            textBox3.Text = "";ip
            textBox6.Text = "8090";
            comboBox2.Text = "http";
            comboBox1.Text = e.server_mode;
            comboBox4.Text = "";
            textBox7.Text = "";
        }

        Writejson(index, configs, jsonconfigpath);
        InitConfig();
    }
    private void ListBox1_MouseDown(object sender, MouseEventArgs e)
    {
        if (e.Button == MouseButtons.Right)
        {
            int posindex = listBox1.IndexFromPoint(new Point(e.X, e.Y));
            listBox1.ContextMenuStrip = null;
            if (posindex >= 0 && posindex < listBox1.Items.Count)
            {
                listBox1.SelectedIndex = posindex;
                contextMenuStrip1.Show(listBox1, new Point(e.X, e.Y));
            }
        }
    }
    private void Button6_Click(object sender, EventArgs e)
    {
        ConfigsItem configsItem = new ConfigsItem();
        configsItem.server = textBox3.Text;//ip
        configsItem.server_port = Convert.ToInt32(textBox6.Text);
        configsItem.server_type = comboBox2.Text;
        configsItem.server_mode = comboBox1.Text;
        configsItem.method = comboBox4.Text;
        configsItem.password = textBox7.Text;
        configs[mainIndex]=configsItem;
        int index = mainIndex;
        Writejson(index, configs, jsonconfigpath);
        InitConfig();
    }
}
}
using System;
using System.Threading.Tasks;

```

```

using commontest.src.test;
namespace clienttest
{
    class Program
    {
        static void RunAsync()
        {
            Action<Int32> action = async (Int32 number) =>
            {
                await Task.Run(() =>
                {
                    try
                    {
                        Console.WriteLine("thread start number:" + number + ",
current thread:" + System.Threading.Thread.CurrentThread.GetHashCode());
                        ICalc proxy =
sharpsvr.proxy.ProxyGenerator.Of<ICalc>(new sharpsvr.proxy.SharpInterceptor());
                        while (true)
                        {
                            try
                            {
                                Console.WriteLine("hello:" + proxy.TestA(1,
2L));
                                var userList = proxy.GetUserList(new User { Id =
123 });
                                foreach (var user in userList)
                                Console.WriteLine("hello:" + user);
                                Console.WriteLine("proxy.GetUser:" +
proxy.GetUser(new User { Id = 1 }, new User { Age = 2 }));
                            }
                            catch (Exception ex)
                            {
                                Console.WriteLine("error exception:" + ex);
                            }
                        }
                    }
                    finally
                    {
                        Console.WriteLine("exception not caught!");
                    }
                });
            };
            for (int i = 0; i < 100; ++i)
            {
                action(i);
            }
        }
        static void RunSync()
        {
            try

```

```

        {
            ICalc proxy = sharpsvr.proxy.ProxyGenerator.Of<ICalc>(new
sharpsvr.proxy.SharpInterceptor());
            while (true)
            {
                try
                {
                    Console.WriteLine("hello:" + proxy.TestA(1, 2L));
                    var userList = proxy.GetUserList(new User { Id = 123 });
                    foreach (var user in userList) Console.WriteLine("hello:"
+ user);

                    Console.WriteLine("proxy.GetUser:" + proxy.GetUser(new
User { Id = 1 }, new User { Age = 2 }));
                }
                catch (Exception ex)
                {
                    Console.WriteLine("error exception:" + ex);
                }
            }
        }
        finally
        {
            Console.WriteLine("exception not caught!");
        }
    }

    static void Main(string[] args)
    {
        RunAsync();
        Console.ReadKey();
        //while(true) System.Threading.Thread.Sleep(100000);
        Environment.Exit(0);
    }
}

using System;
namespace commontest
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}

using System.Collections.Generic;
namespace commontest.src.test
{
    public class Calc : ICalc
    {

```



```

        public double Add(int a, long b, float c = 1, double d = 2)
        {
            return a + b + c + d;
        }
        public User GetUser(User a = null, User b = null)
        {
            System.Console.WriteLine($"a={a}, b={b}");
            throw new System.NotImplementedException();
        }
        public List<User> GetUserList(User user = null)
        {
            var result = new List<User>();
            result.Add(user);
            result.Add(new User{Id=1, Age=2, Sex=true, Message = "hello,hahah",
Child=user});
            return result;
        }
        public void SayHello(string name = "yangyang")
        {
            System.Console.WriteLine("hello:" + name);
        }
        public int TestA(short b)
        {
            return b++;
        }
        public int TestA(int c, long d)
        {
            return (int)(c * d);
        }
    }
}

using System.Collections.Generic;
using sharpsvr.attributes;
namespace commontest.src.test
{
    [RemoteService]
    public interface ICalc
    {
        [RemoteMethod]
        void SayHello(string name="yangyang");
        [RemoteMethod]
        int TestA(short b);
        [RemoteMethod]
        int TestA(int c, long d);
        [RemoteMethod]
        double Add(int a, long b, float c=1.0f, double d = 2.0);
        [RemoteMethod]
        List<User> GetUserList(User user = null);
        [RemoteMethod]
        User GetUser(User a=null, User b=null);
    }
}

```

```
    }
}
using System;
namespace commontest.src.test
{
    [Serializable]
    public class User
    {
        public long Id { get; set; }
        public int Age { get; set; }
        public bool Sex { get; set; }
        public string Message { get; set; }
        public User Child { get; set; }

        public override string ToString() {
            return $"Id={Id}, Age={Age}, Sex={Sex}, Message={Message},
Child={Child}";
        }
    }
}
using System;
using System.IO;
using System.Reflection;
using System.Reflection.Emit;
using System.Threading.Tasks;
using log4net;
using sharpsvr.attributes;
using sharpsvr.common;
using sharpsvr.core;
using sharpsvr.net;
using sharpsvr.proxy;
namespace sharpsvr
{
    class Program
    {
        static ILog log = Logger.GetInstance().GetLog();
        class A { }
        static void RunServer()
        {
            var server = new TcpServer();
            server.Startup();
            Action<IServer> MainLoop = async (IServer svr) =>
            {
                await Task.Run(() => { svr.MainLoop(); });
            };
            MainLoop(server);
            Console.ReadKey();
            server.ShutDown();
        }
    }
}
```

```

        static void TestSerialize() {
            var packet = new protocol.SharpPacket{
                Random=1,
                Version = "12.0.32",
                MethodUniqueName = "test",
                ServiceUniqueName = "te",
                Result = new Exception("test"),
                Arguments = new object[] {1, "2", null}
            };
            var serialize =
protocol.SerializerManager.GetInstance().GetSerializer(SharpSvrSettings.GetInst
ance().EncoderName);
            var bytes = serialize.Serialize(packet);
            log.Debug("deserialize : " +
serialize.Deserialize<protocol.SharpPacket>(bytes));
        }
        public static void Main(string[] args)
        {
            TestSerialize();
        }
    }
}
using System;
namespace sharpsvr.attributes
{
    [AttributeUsage(AttributeTargets.Method, AllowMultiple=false,
Inherited=false)]
    public class RemoteMethodAttribute : Attribute
    {
        public string Name {get;set;} = null;
    }
}
using System;
namespace sharpsvr.attributes
{
    [AttributeUsage(AttributeTargets.Interface|AttributeTargets.Class,
AllowMultiple=false, Inherited = false)]
    public class RemoteServiceAttribute : Attribute
    {
        public string Name {get;set;} = null;
        public string Version{get;set;} = "1.0.0";
    }
}
using System;
using System.IO;
using log4net;
using log4net.Config;
using log4net.Repository;
namespace sharpsvr.common
{
    public static class LogHelper

```

```
{
    private enum LogLevelEnum
    {
        Debug, Info, Warn, Error, Fatal
    }

    private static void Log(this ILog log, LogLevelEnum levelEnum, string
format, params object[] args)
    {
        switch (levelEnum)
        {
            case LogLevelEnum.Debug:
                if (log.IsDebugEnabled)
                {
                    string message = string.Format(format, args);
                    log.Debug(message);
                }
                break;
            case LogLevelEnum.Info:
                if (log.IsInfoEnabled)
                {
                    string message = string.Format(format, args);
                    log.Info(message);
                }
                break;
            case LogLevelEnum.Warn:
                if (log.IsWarnEnabled)
                {
                    string message = string.Format(format, args);
                    log.Warn(message);
                }
                break;
            case LogLevelEnum.Error:
                if (log.IsErrorEnabled)
                {
                    string message = string.Format(format, args);
                    log.Error(message);
                }
                break;
            case LogLevelEnum.Fatal:
                if (log.IsFatalEnabled)
                {
                    string message = string.Format(format, args);
                    log.Fatal(message);
                }
                break;
        }
    }

    public static void LogDebug(this ILog log, string format, params object[]
args)
    {

```

```

        Log(log, LogLevelEnum.Debug, format, args);
    }
    public static void LogInfo(this ILog log, string format, params object[]
args)
    {
        Log(log, LogLevelEnum.Info, format, args);
    }
    public static void LogWarn(this ILog log, string format, params object[]
args)
    {
        Log(log, LogLevelEnum.Warn, format, args);
    }
    public static void LogError(this ILog log, string format, params object[]
args)
    {
        Log(log, LogLevelEnum.Error, format, args);
    }
    public static void LogFatal(this ILog log, string format, params object[]
args)
    {
        Log(log, LogLevelEnum.Fatal, format, args);
    }
}
public class Logger : Singleton<Logger>
{
    private          ILoggerRepository          respository          =
LogManager.CreateRepository("sharpsvr");
    public Logger()
    {
        var path = PathUtils.GetAppConfigFilePath();
        if (File.Exists("app.config"))
        {
            Console.WriteLine("use app.config to config log4net.");
            log4net.Config.XmlConfigurator.Configure(respository,          new
FileInfo("app.config"));
        }
        else
        {
            Console.WriteLine("app.config not found! use default log4net
settings.");
            BasicConfigurator.Configure(respository);
        }
    }
    public ILog GetLog(string name)
    {
        return LogManager.GetLogger(respository.Name, name);
    }
    public ILog GetLog(Type type)
    {
        return LogManager.GetLogger(respository.Name, type);
    }
}

```

```
    }
    public ILog GetLog()
    {
        return GetLog("sharpsvr");
    }
}

using System;
using System.IO;
namespace sharpsvr.common
{
    public static class PathUtils
    {
        public static string GetCurrentProjectPath()
        {
            var path = Environment.CurrentDirectory;
            if (path.IndexOf("bin") > 0)
            {
                path = path.Substring(0, path.IndexOf("bin"));
            }
            return path;
        }
        public static string GetAppConfigFilePath() {
            return Path.Combine(GetCurrentProjectPath(), "app.config");
        }
    }
}

using System;
using System.Collections.Generic;
using System.IO;
using System.Reflection;
using log4net;
using Microsoft.Extensions.Configuration;
using sharpsvr.util;
namespace sharpsvr.common
{
    [AttributeUsage(AttributeTargets.Property, AllowMultiple = false, Inherited = false)]
    class BindPropertyAttribute : Attribute
    {
        public string name;
        public object value;
    }
    public class ServerConfig
    {
        public string ServerName { get; set; }
        public string ServerIp { get; set; }
        public short ServerPort { get; set; } = 10086;
        public byte Weight { get; set; } = 1;
        public override string ToString()
    }
}
```

```

        {
            return $"ServerName={ServerName}, ServerIp={ServerIp},
ServerPort={ServerPort}, Weight={Weight}";
        }
    }

    public class ServiceConfig
    {
        public string ServiceName { get; set; }
        public string ServiceVersion { get; set; }
        public string TypeName { get; set; }
        public List<ServerConfig> ServerList = new List<ServerConfig>();
        public override String ToString()
        {
            return $"ServiceName={ServiceName}, ServiceVersion={ServiceVersion},
TypeName={TypeName}, serverList={string.Join(',', ServerList)}";
        }
    }

    public class SharpSvrSettings : Singleton<SharpSvrSettings>
    {
        private static ILog log = Logger.GetInstance().GetLog();
        public SharpSvrSettings()
        {
            IConfigurationSection configuration = null;
            var path = PathUtils.GetAppConfigFilePath();
            if (File.Exists(path))
            {
                log.Info("use app.config file to set properties. path:" + path);
                var builder = new
ConfigurationBuilder().SetBasePath(Directory.GetCurrentDirectory()).AddXmlFile(
path);

                configuration = builder.Build().GetSection("sharpsvr");
            }
            else
            {
                log.Warn("app.config file not found! use default settings.");
                return;
            }
            Type type = GetType();
            var properties = type.GetProperties();
            foreach (var property in properties)
            {
                var attribute = =
AttributeUtils.GetCustomAttribute<BindPropertyAttribute>(property);
                if (attribute != null)
                {
                    Object value = configuration != null ?
configuration[attribute.name] : null;
                    value = Convert.ChangeType(value ?? attribute.value,
property.PropertyType);
                    property.SetValue(this, value ?? attribute.value);
                }
            }
        }
    }

```

```

    }
}
Name2AllService = new Dictionary<string, ServiceConfig>();
Type2AllService = new Dictionary<Type, ServiceConfig>();
var remoteServiceSection =
configuration.GetSection("remote-services");
if (remoteServiceSection != null)
{
    var serviceSescions = remoteServiceSection.GetChildren();
    foreach (var middleSection in serviceSescions)
    {
        foreach (var serviceSection in middleSection.GetChildren())
        {
            var serviceConfig = new ServiceConfig
            {
                ServiceName = serviceSection.Key,
                ServiceVersion =
string.IsNullOrEmpty(serviceSection["version"]) ? "1.0.0" :
serviceSection["version"],
                TypeName = serviceSection["type"]
            };
            var serverSection =
serviceSection.GetSection("servers");
            if (serverSection == null) continue;
            foreach (var subMiddleSection in
serverSection.GetChildren())
            {
                foreach (var childSection in
subMiddleSection.GetChildren())
                {
                    serviceConfig.ServerList.Add(new ServerConfig
                    {
                        ServerName = childSection.Key,
                        ServerIp = childSection["ip"],
                        ServerPort =
short.Parse(childSection["port"]),
                        Weight =
(byte)(string.IsNullOrEmpty(childSection["weight"]) ? 1 :
byte.Parse(childSection["weight"]))
                    });
                }
            }
            if (serviceConfig.ServiceName != null)
            {
                log.Warn("found service:" + serviceConfig);
                Name2AllService.Add(serviceConfig.ServiceName,
serviceConfig);
                foreach (var assembly in
AppDomain.CurrentDomain.GetAssemblies())
            {

```



```

        if (assembly.GetType(serviceConfig.TypeName) !=
null)
        {
Type2AllService.Add(assembly.GetType(serviceConfig.TypeName), serviceConfig);
            break;
        }
    }
}

public string EncoderName { get; private set; } = "json";
public uint ByteBufferInitSize { get; private set; }
[BindProperty(name = "session.sendbuffer.size", value = 1024)]
public uint SessionSendBufferInitSize { get; private set; }
[BindProperty(name = "session.recvbuffer.size", value = 1024)]
public uint SessionRecvBufferInitSize { get; private set; }
[BindProperty(name = "session.sendrecv.per.time", value = 1024)]
public uint SendRecvPerTime { get; private set; }
[BindProperty(name = "server.bind.ip", value = "127.0.0.1")]
public string BindIp { get; private set; }
[BindProperty(name = "client.connect.ip", value = "127.0.0.1")]
public string ConnectIp { get; private set; }
[BindProperty(name = "server.listen.port", value = 10086)]
public short ListenPort { get; private set; }
[BindProperty(name = "client.connect.port", value = 10086)]
public short ConnectPort { get; private set; }
[BindProperty(name = "client.recv.timeout", value = 1000000)]
public int ClientRecvTimeout { get; private set; }
[BindProperty(name = "client.sendrecv.time.unit", value = 100)]
public int ClientSendRecvTimeUnit { get; private set; }
[BindProperty(name = "server.listen.backlog", value = 128)]
public uint ListenBacklog { get; private set; }
[BindProperty(name = "server.select.microsecond", value = 100)]
public uint SelectMicroSeconds { get; private set; }
[BindProperty(name = "server.shutdown.try.count", value = 10)]
public uint ShutDownTryCount { get; private set; }
[BindProperty(name = "server.udp.recv.timeout", value = -1)]
public int UdpRecvTimeOut { get; private set; }
[BindProperty(name = "sharpsvr.serialize.type", value = "json")]
public string SerializerType { get; private set; }
[BindProperty(name = "server.main.sleep.time", value = 1)]
public int ServerMainSleepTime { get; private set; }
[BindProperty(name = "client.keepalive", value = true)]
public bool ClientKeepAlive { get; private set; } = true;
[BindProperty(name = "client.max.connect.fail.count", value = 10)]
public int MaxConnectFailCount { get; private set; }

```

```
        public Dictionary<String, ServiceConfig> Name2AllService { get; private
set; }
        public Dictionary<Type, ServiceConfig> Type2AllService { get; private
set; }
    }
}
namespace sharpsvr.common
{
    public class Singleton<T> where T : new()
    {
        private static T instance;
        public static T GetInstance()
        {
            if (instance != null) return instance;
            lock (typeof(Singleton<T>))
            {
                if (instance == null)
                {
                    instance = new T();
                }
            }
            return instance;
        }
    }
}
using log4net;
using sharpsvr.common;
namespace sharpsvr.core
{
    public class ByteArrayFactory : Singleton<ByteArrayFactory>
    {
        private static ILog log = Logger.GetInstance().GetLog();
        public byte[] allocate(uint size) {
            log.Debug($"allocating byte buffer: {size}");
            return new byte[size];
        }
        public void recycle(byte[] bytes) {
            log.Debug($"recyle byte buffer {bytes.Length}");
        }
    }
}
using System;
using log4net;
using sharpsvr.common;
namespace sharpsvr.core
{
    using UInt = UInt32;
    [Serializable]
    public class ByteBuffer : ICloneable
    {
```

```

private static ILog log = Logger.GetInstance().GetLog();
private byte[] buf;
public UInt ReadPosition { get; set; } = 0;

public UInt Used { get; set; } = 0;
public UInt Free { get { return (UInt)buf.Length - Used; } }
public UInt WritePosition { get { return (UInt)((ReadPosition + Used) %
buf.Length); } }
public ByteBuffer(UInt bytes = 0)
{
    buf = ByteArrayFactory.GetInstance().allocate(bytes > 0 ? bytes :
common.SharpSvrSettings.GetInstance().ByteBufferInitSize);
    log.Debug(String.Format("new    bytebuffer    created.    size={0}",
buf.Length));
}
public void EnsureEnoughFreeSize(UInt size)
{
    if (Free >= size) return;
    UInt newSize = (UInt)(buf.Length << 1);
    while (newSize - Used < size) newSize = newSize << 1;
    byte[] newBuf = ByteArrayFactory.GetInstance().allocate(newSize);
    log.LogDebug("allocate buffer size: from {0} to {1}", buf.Length,
newSize);
    if (ReadPosition < WritePosition)
    {
        Array.Copy(buf, ReadPosition, newBuf, 0, Used);
    }
    else if (Used > 0)
    {
        Array.Copy(buf, ReadPosition, newBuf, 0, buf.Length -
ReadPosition);
        Array.Copy(buf, 0, newBuf, buf.Length - ReadPosition,
WritePosition);
    }
    ByteArrayFactory.GetInstance().recycle(buf);
    buf = newBuf;
    ReadPosition = 0;
}
public UInt WriteBytes(byte[] bytes, UInt from = 0, UInt to = 0)
{
    if (bytes == null || from > to) return 0;
    if (to > bytes.Length || to == 0) to = (UInt)bytes.Length;
    UInt copySize = to - from;
    if (copySize == 0) return 0;
    EnsureEnoughFreeSize(copySize);
    if (buf.Length - ReadPosition - Used >= copySize || WritePosition <
ReadPosition)
    {
        Array.Copy(bytes, from, buf, WritePosition, copySize);
    }
}

```

```
        else
        {
            UInt firstCopySize = (UInt)(buf.Length - WritePosition);
            Array.Copy(bytes, from, buf, WritePosition, firstCopySize);
            Array.Copy(bytes, from + firstCopySize, buf, 0, copySize -
firstCopySize);
        }
        Used += copySize;
        return copySize;
    }

    public UInt ReadBytes(ref byte[] bytes, UInt len = 0)
    {
        if (bytes != null && len == 0) len = (UInt)bytes.Length;
        if (bytes == null)
        {
            if (len == 0) len = Used;
            bytes = ByteArrayFactory.GetInstance().allocate(len);
        }
        len = (UInt)Math.Min(Math.Min(len, bytes.Length), Used);
        if (ReadPosition + len <= buf.Length)
        {
            Array.Copy(buf, ReadPosition, bytes, 0, len);
        }
        else
        {
            UInt firstCopySize = (UInt)(buf.Length - ReadPosition);
            Array.Copy(buf, ReadPosition, bytes, 0, firstCopySize);
            Array.Copy(buf, 0, bytes, buf.Length - ReadPosition, len -
firstCopySize);
        }
        Used -= len;
        ReadPosition = (UInt)((ReadPosition + len) % buf.Length);
        return len;
    }

    public UInt PeekBytes(ref byte[] bytes, UInt len = 0)
    {
        if (bytes != null && len == 0) len = (UInt)bytes.Length;
        if (bytes == null)
        {
            if (len == 0) len = Used;
            bytes = ByteArrayFactory.GetInstance().allocate(len);
        }
        len = (UInt)Math.Min(Math.Min(len, bytes.Length), Used);
        if (ReadPosition + len <= buf.Length)
        {
            Array.Copy(buf, ReadPosition, bytes, 0, len);
        }
        else
        {
            UInt firstCopySize = (UInt)(buf.Length - ReadPosition);
```

```
        Array.Copy(buf, ReadPosition, bytes, 0, firstCopySize);
        Array.Copy(buf, 0, bytes, buf.Length - ReadPosition, len -
firstCopySize);
    }
    return len;
}

public UInt WriteString(String str)
{
    var bytes = System.Text.Encoding.UTF8.GetBytes(str);
    if (bytes.Length > UInt16.MaxValue - 2)
    {
        log.Error($"str too long to write! Len={bytes.Length}");
        return 0;
    }
    var sizeLen = WriteUInt16((UInt16)bytes.Length);
    return sizeLen + WriteBytes(bytes);
}

public UInt ReadString(out String str)
{
    UInt16 size;
    var sizeLen = ReadUInt16(out size);
    if (sizeLen != 2)
    {
        log.Error($"string read failed! length size={sizeLen}");
        str = null;
        return 0;
    }
    byte[] buffer = null;
    sizeLen += ReadBytes(ref buffer, size);
    str = System.Text.Encoding.UTF8.GetString(buffer);
    ByteArrayFactory.GetInstance().recycle(buffer);
    return sizeLen;
}

public UInt WriteUInt16(UInt16 value)
{
    byte[] buffer = { (byte)(value & 0xff), (byte)(value >> 8) };
    return WriteBytes(buffer);
}

public UInt ReadUInt16(out UInt16 value)
{
    byte[] buffer = null;
    if (Used >= 2)
    {
        UInt len = ReadBytes(ref buffer, 2);
        value = (UInt16)((buffer[1] << 8) + buffer[0]);
        return len;
    }
    else
    {
        value = 0;
    }
}
```

```
        return 0;
    }
}

public UInt WriteUInt32(UInt32 value)
{
    byte[] buffer = { (byte)(value & 0xff), (byte)((value >> 8) & 0xff),
(byte)((value >> 16) & 0xff), (byte)((value >> 24) & 0xff) };
    return WriteBytes(buffer);
}

public UInt ReadUInt32(out UInt32 value)
{
    byte[] buffer = null;
    if (Used >= 4)
    {
        UInt len = ReadBytes(ref buffer, 4);
        value = (UInt32)((UInt32)(buffer[3] << 24) + (UInt32)(buffer[2] <<
16) + (UInt32)(buffer[1] << 8) + (UInt32)buffer[0]);
        return len;
    }
    else
    {
        value = 0;
        return 0;
    }
}

public UInt WriteUInt64(UInt64 value)
{
    byte[] buffer = {
        (byte)(value & 0xff),
        (byte)((value >> 8) & 0xff),
        (byte)((value >> 16) & 0xff),
        (byte)((value >> 24) & 0xff),
        (byte)((value >> 32) & 0xff),
        (byte)((value >> 40) & 0xff),
        (byte)((value >> 48) & 0xff),
        (byte)((value >> 56) & 0xff)
    };
    return WriteBytes(buffer);
}

public UInt ReadUInt64(out UInt64 value)
{
    byte[] buffer = null;
    if (Used >= 8)
    {
        UInt len = ReadBytes(ref buffer, 8);
        value = (((UInt64)buffer[3]) << 24) + ((UInt64)buffer[2] << 16) +
((UInt64)buffer[1] << 8) + buffer[0] +
        (((UInt64)buffer[7]) << 56) + ((UInt64)buffer[6] << 48) +
((UInt64)buffer[5] << 40) + ((UInt64)buffer[4] << 32);
        return len;
    }
}
```

```

        }
        else
        {
            value = 0;
            return 0;
        }
    }

    public byte this[int index] { get { return buf[(ReadPosition + index) %
buf.Length]; } }
    public object Clone()
    {
        ByteBuffer byteBuffer = new ByteBuffer((UInt)this.buf.Length);
        byteBuffer.ReadPosition = this.ReadPosition;
        byteBuffer.Used = this.Used;
        Array.Copy(this.buf, byteBuffer.buf, this.buf.Length);
        return byteBuffer;
    }
    public override String ToString()
    {
        return    $"Buf.Length={buf.Length},    ReadPosition={ReadPosition},
Used={Used}";
    }
}
}
using System;
using System.Runtime.Serialization;
namespace sharpsvr.exception
{
    [Serializable]
    public class SharpServerException : SystemException
    {
        public SharpServerException() : base() {}
        public SharpServerException(string message) : base(message) {}
        public SharpServerException(string message, Exception ex) : base(message,
ex) {}
        protected SharpServerException(SerializationInfo info, StreamingContext
context):base(info, context) {}
    }
}
using System;
namespace sharpsvr.exception
{
    [Serializable]
    public class WrapServerException
    {
        public string Message{get;set;}
        public WrapServerException(string message)
        {
            Message = message;
        }
    }
}

```

```

        public override string ToString() {
            return Message;
        }
    }
}
namespace sharpsvr.net
{
    public class ClientSocketSession : SocketSession
    {

    }
}
namespace sharpsvr.net
{
    public interface IServer
    {
        void StartUp(string ip=null, short port=0);
        void MainLoop();
        void ShutDown();
    }
}
namespace sharpsvr.net
{
    public class ServerSocketSession : SocketSession
    {

    }
}
using System;
using System.Collections.Generic;
using System.Reflection;
using sharpsvr.attributes;
using sharpsvr.common;
using sharpsvr.protocol;
using sharpsvr.util;
namespace sharpsvr.net
{
    public class SharpServer : TcpServer
    {
        private Dictionary<string, Tuple<string, MethodInfo, object>> proxy;
        private log4net.ILog log = Logger.GetInstance().GetLog();
        public SharpServer(params object[] objects)
        {
            proxy = new Dictionary<string, Tuple<string, MethodInfo, object>>();
            WithService(objects);
        }
        public void ScanTypeProxy(Type type, object obj)
        {
            var serviceAttribute =
AttributeUtils.GetRemoteServiceAttribute(type);

```



```

        if (serviceAttribute == null) return;
        var serviceName = serviceAttribute.Name;
        if (string.IsNullOrEmpty(serviceName)) serviceName =
TypeUtils.GetTypeUniqueName(type);
        if (string.IsNullOrEmpty(serviceName))
        {
            log.Error($"type name for {type} not defined!");
            return;
        }
        log.Info($"scan service {serviceName} for type {type.FullName}");
        foreach (var methodInfo in type.GetMethods())
        {
            var methodName = TypeUtils.GetMethodUniqueName(methodInfo);
            if (string.IsNullOrEmpty(methodName))
            {
                log.Error($"method name for {methodInfo} not defined!");
                continue;
            }
            var fullName = TypeUtils.FullMethodName(serviceName, methodName);
            if (proxy.ContainsKey(fullName))
            {
                log.Error($"found method proxy {fullName} more than once,
ignored!");
                continue;
            }
            proxy.Add(fullName, new Tuple<string, MethodInfo,
object>(serviceAttribute.Version, methodInfo, obj));
            log.Info($"found method {fullName}.");
        }
        log.Info($"scan service {serviceName} for type {type.FullName}
success!");
    }

    public SharpServer WithService(params object[] objects)
    {
        lock (proxy)
        {
            foreach (var obj in objects)
            {
                if (obj == null) continue;
                var type = obj.GetType();
                var interfaces = type.GetInterfaces();
                if (interfaces != null && interfaces.Length > 0)
                {
                    foreach (var interfaceType in interfaces)
                    {
                        ScanTypeProxy(interfaceType, obj);
                    }
                }
                ScanTypeProxy(type, obj);
            }
        }
    }

```

```

    }
    return this;
}
protected override void OnStartup(string ip, short port)
{
    base.OnStartup(ip, port);
    this.RecvData += this.TryDecodePacket;
}
private void TryDecodePacket(SocketSession session)
{
    var packet =
protocol.SharpProtocol.GetInstance().TryDecodePacket(session);
    if (packet != null)
    {
        ProcessClientPacket(session, packet);
    }
}
private void ProcessClientPacket(SocketSession session, SharpPacket
packet)
{
    log.Debug("request detected:" + packet);
    var fullName = TypeUtils.FullMethodName(packet.ServiceUniqueName,
packet.MethodUniqueName);
    if (proxy.ContainsKey(fullName))
    {
        var methodInfo = proxy[fullName];
        if (!methodInfo.Item1.Equals(packet.Version))
        {
            log.Warn($"packet version={packet.Version} and server
version={methodInfo.Item1} not equals!");
        }
        try
        {
            packet.Result = methodInfo.Item2.Invoke(methodInfo.Item3,
ConvertUtils.ConvertObjectTypes(methodInfo.Item2, packet.Arguments,
packet.Encoder));
        }
        catch (TargetInvocationException ex)
        {
            log.Error("invoke method:" + packet.MethodUniqueName + "
error!", ex);
            packet.Result =
new
exception.WrapServerException(ex.InnerException.ToString());
        } catch (Exception ex) {
            log.Error("invoke method:" + packet.MethodUniqueName + "
error!", ex);
            packet.Result =
new
exception.WrapServerException(ex.Message.ToString());
        }
    }
    var resultByteArray =

```

```

protocol.SharpProtocol.GetInstance().Encode(packet);

session.Send(protocol.SharpProtocol.GetHeader(resultByteArray));
    session.Send(resultByteArray);
    }
    }
    }
}

using System;
using System.Net;
using System.Net.Sockets;
using System.Reflection;
using System.Threading;
using sharpsvr.common;
using sharpsvr.exception;
namespace sharpsvr.net
{
    public class SocketClient
    {
        private static log4net.ILog log = Logger.GetInstance().GetLog();
        public Int32 ConnectFailCount;
        protected SocketSession clientSession;
        public SocketClient(string ip = "127.0.0.1", short port = 10086)
        {
            clientSession = new ClientSocketSession();
        }
        public void Connect(string ip = "127.0.0.1", short port = 10086)
        {
            if(clientSession.IsConnected) return;
            if
(!string.IsNullOrEmpty(SharpSvrSettings.GetInstance().ConnectIp))
            {
                if ("127.0.0.1".Equals(ip)) ip =
SharpSvrSettings.GetInstance().ConnectIp;
            }
            if (SharpSvrSettings.GetInstance().ConnectPort > 0)
            {
                if (port == 10086) port =
SharpSvrSettings.GetInstance().ConnectPort;
            }
            EndPoint endPoint = new IPEndPoint(IPAddress.Parse(ip), port);
            var socket = new Socket(AddressFamily.InterNetwork, SocketType.Stream,
ProtocolType.Tcp);
            socket.SendTimeout = socket.ReceiveTimeout =
SharpSvrSettings.GetInstance().ClientSendRecvTimeUnit;
            try{
                socket.Connect(endPoint);
                clientSession.WithSocket(socket);
                Interlocked.Exchange(ref ConnectFailCount, 0);
            }catch(SocketException se){

```

```

        Interlocked.Add(ref ConnectFailCount, 1);
        throw se;
    }
}

public void Disconnect()
{
    if(!clientSession.IsConnected) return;
    clientSession.Close();
}

public protocol.SharpPacket SendAndRecvPacket(MethodInfo methodInfo,
object[] args)
{
    var contentBytes =
protocol.SharpProtocol.GetInstance().Encode(methodInfo, args);
    clientSession.Send(protocol.SharpProtocol.GetHeader(contentBytes));
    clientSession.Send(contentBytes);
    var timeOutTime = SharpSvrSettings.GetInstance().ClientRecvTimeout;
    var waitTime = 0L;
    while (waitTime < timeOutTime)
    {
        long start = DateTime.Now.Millisecond;
        if (clientSession.BindSocket.Poll(10000, SelectMode.SelectRead |
SelectMode.SelectWrite))
        {
            clientSession.CanRead = true;
            clientSession.CanWrite = true;
            clientSession.DoSend();
            clientSession.DoRecv();
            if (clientSession.IsError)
            {
                clientSession.Close();
                string message = $"server error occurred! when invoke method
{methodInfo.Name} with arguments {args}";
                log.Error(message);
                return new protocol.SharpPacket { Result = new
WrapServerException(message) };
            }
            var packet =
protocol.SharpProtocol.GetInstance().TryDecodePacket(clientSession);
            if (packet != null)
            {
                //log.Debug("recv packet:" + packet.Random);
                return packet;
            }
            waitTime = (DateTime.Now.Millisecond - start) * 1000;
        }
    }
    return null;
}
}

```

```

}
using System;
using System.Collections;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Net;
using System.Net.Sockets;
using System.Threading;
using sharpsvr.common;
namespace sharpsvr.net
{
    public abstract class SocketServer : IServer
    {
        private static log4net.ILog log = Logger.GetInstance().GetLog();
        protected SocketSession serverSession;
        protected ConcurrentDictionary<Socket, SocketSession> clientSession;
        private volatile bool isRunning;
        protected HashSet<SocketSession> readWriteSessionList;
        protected abstract void OnConnected(SocketSession session);
        protected abstract void OnDisConnected(SocketSession session);
        protected abstract void OnSendData(SocketSession session, uint len);
        protected abstract void OnRecvData(SocketSession session, uint len);
        protected abstract void BeforeSendData(SocketSession session, byte[] bytes,
uint from, uint to);
        protected abstract void BeforeRecvData(SocketSession session, byte[] bytes,
uint from, uint to);
        protected abstract void OnStartUp(string ip, short port);
        protected abstract void OnMainLoop();
        protected abstract void OnShutDown();
        public SocketServer()
        {
            clientSession = new ConcurrentDictionary<Socket, SocketSession>();
            serverSession = new ServerSocketSession();
            readWriteSessionList = new HashSet<SocketSession>();
        }
        public void StartUp(string ip = null, short port = 0)
        {
            if (string.IsNullOrEmpty(ip)) ip =
SharpSvrSettings.GetInstance().BindIp;
            if (port <= 0) port = SharpSvrSettings.GetInstance().ListenPort;
            OnStartUp(ip, port);
            isRunning = true;
        }
        public void ShutDown()
        {
            log.Warn("going to shut down socket server...");
            isRunning = false;
            OnShutDown();
            isRunning = false;
            log.Warn("socket server shutdown success!");
        }
    }
}

```

```

    }
    public void MainLoop()
    {
        long runLoop = 0;
        while (isRunning)
        {
            if (runLoop++ % 10000 == 0)
            {
                log.Debug("socket server is running..., active clients:" +
clientSession.Count);
            }
            OnMainLoop();
            if (SharpSvrSettings.GetInstance().ServerMainSleepTime > 0)
Thread.Sleep(SharpSvrSettings.GetInstance().ServerMainSleepTime);
        }
    }
    protected void AddClientSocket(Socket clientSocket)
    {
        if (clientSocket == null) return;
        if (!clientSession.ContainsKey(clientSocket))
        {
            var session = new ClientSocketSession().WithSocket(clientSocket);
            clientSession[clientSocket] = session;
            OnConnected(session);
            session.OnSend += (ref byte[] bytes, ref uint from, ref uint to)
=>
            {
                BeforeSendData(session, bytes, from, to);
            };
            session.OnRecv += (ref byte[] bytes, ref uint from, ref uint to)
=>
            {
                BeforeRecvData(session, bytes, from, to);
            };
        }
    }
    protected void RemoveClientSocket(Socket clientSocket)
    {
        if (clientSocket == null) return;
        SocketSession session = null;
        int maxRetryCount = 10;
        bool result = false;
        while(maxRetryCount-- > 0 && !result){
            result = clientSession.Remove(clientSocket, out session);
            if (session != null) OnDisConnected(session);
        }
    }
    public void WaitCallback(object state)
    {
        SocketSession session = state as SocketSession;

```

```

        if (session == null) return;
        lock (session)
        {
            var len = session.DoSend();
            if (len > 0)
            {
                OnSendData(session, (uint)len);
            }
            len = session.DoRecv();
            if (len > 0)
            {
                OnRecvData(session, (uint)len);
            }
        }
    }
}

using System;
using System.Net;
using System.Net.Sockets;
using sharpsvr.common;
using sharpsvr.core;
namespace sharpsvr.net
{
    public abstract class SocketSession
    {
        public delegate void ByteArrayTransactionDelegate(ref byte[] bytes, ref
uint from, ref uint to);
        private static log4net.ILog log = Logger.GetInstance().GetLog();
        public Socket BindSocket { get; private set; }
        public ByteBuffer SendBuffer { get; private set; }
        public ByteBuffer RecvBuffer { get; private set; }
        private byte[] sendRecvPerTime =
ByteArrayFactory.GetInstance().allocate(SharpSvrSettings.GetInstance().SendRecv
PerTime);
        public bool IsConnected { get { return BindSocket != null ?
BindSocket.Connected : false; } }
        public volatile bool IsError;
        public event ByteArrayTransactionDelegate OnSend;
        public event ByteArrayTransactionDelegate OnRecv;
        public bool CanRead { get; set; }
        public bool CanWrite { get; set; }
        public SocketSession()
        {
            SendBuffer =
new
ByteBuffer(SharpSvrSettings.GetInstance().SessionSendBufferInitSize);
            RecvBuffer =
new
ByteBuffer(SharpSvrSettings.GetInstance().SessionRecvBufferInitSize);
        }
        public SocketSession WithSocket(Socket socket)

```

```

    {
        this.BindSocket = socket;
        //socket.SetSocketOption(SocketOptionLevel.IP, )
        return this;
    }
    public int DoSend()
    {
        if (IsConnected && SendBuffer.Used > 0 && CanWrite)
        {
            var len = SendBuffer.ReadBytes(ref sendRecvPerTime,
            (uint)sendRecvPerTime.Length);
            uint from = 0, to = len;
            if (OnSend != null) OnSend(ref sendRecvPerTime, ref from, ref to);
            int send = 0;
            len = to - from;
            while (send < len) send += BindSocket.Send(sendRecvPerTime,
            (int)from + send, (int)(len - send), SocketFlags.None);
            CanWrite = false;
            return send;
        }
        return 0;
    }
    public int DoRecv()
    {
        try
        {
            if (IsConnected && CanRead)
            {
                var len = BindSocket.Receive(sendRecvPerTime, 0,
                sendRecvPerTime.Length, SocketFlags.None);
                uint from = 0, to = (uint)len;
                if (OnRecv != null) OnRecv(ref sendRecvPerTime, ref from, ref
                to);

                len = (int)(to - from);
                if (len > 0) RecvBuffer.WriteBytes(sendRecvPerTime, from, to);
                else{
                    log.Debug($"connection disconnected!
socket={BindSocket.Handle}");
                    IsError = true;
                }
                CanRead = false;
                return len;
            }
            return 0;
        }
        catch (Exception ex)
        {
            if(IsConnected) return 0;
            log.Error("DoRecv error!", ex);
            IsError = true;
        }
    }

```



```

        return -1;
    }
}

public int Send(byte[] bytes, uint from = 0, uint to = 0)
{
    try
    {
        if (bytes == null) return 0;
        if (to == 0) to = (uint)bytes.Length;
        if (from >= to) return 0;
        if (IsConnected)
        {
            SendBuffer.WriteBytes(bytes, from, to);
            return (int)(to - from);
        }
        else
        {
            log.Error("socket is not connected! Data send failed!");
            return 0;
        }
    }
    catch (Exception ex)
    {
        log.Error("DoSend error!", ex);
        IsError = true;
        return -1;
    }
}

public uint DoSendTo(EndPoint endPoint, byte[] bytes, uint from = 0, uint
to = 0) {
    var len = BindSocket.SendTo(bytes, (int) from, (int)(to - from),
SocketFlags.None, endPoint);
    return 0;
}

public uint BroadCast(byte[] bytes, uint from=0, uint to=0) {
    return 0;
}

public uint DoRecvFrom(EndPoint endPoint) {
    uint from = 0, to =(uint) sendRecvPerTime.Length;
    var len = BindSocket.ReceiveFrom(sendRecvPerTime, (int)from, (int)to,
SocketFlags.None, ref endPoint);
    if(len > 0) {
        to = (uint)(from + len);
        if(OnRecv != null) {
            OnRecv(ref sendRecvPerTime, ref from, ref to);
        }
        len = (int)(to - from);
        if( len > 0) {
            RecvBuffer.WriteBytes(sendRecvPerTime, from, to);
        }
    }
}

```

```

        }
        return 0;
    }
    public void Close() {
        if(IsConnected) {
            try{
                BindSocket.Close();
            }finally{
            }
        }
    }
}

using System;
using System.Linq;
using System.Collections;
using System.Collections.Generic;
using System.Net;
using System.Net.Sockets;
using System.Threading;
using sharpsvr.common;
namespace sharpsvr.net
{
    public class TcpServer : SocketServer
    {
        public delegate void SessionEventDelegate(SocketSession session);
        public event SessionEventDelegate Connected;
        public event SessionEventDelegate Disconnected;
        public event SessionEventDelegate SendData;
        public event SessionEventDelegate RecvData;
        private log4net.ILog log = Logger.GetInstance().GetLog();
        protected override void BeforeRecvData(SocketSession session, byte[] bytes,
uint from, uint to)
        {
            log.Debug($"before recv data:{bytes}");
        }
        protected override void BeforeSendData(SocketSession session, byte[] bytes,
uint from, uint to)
        {
            log.Debug($"before send data:{bytes}");
        }
        protected override void OnConnected(SocketSession session)
        {
            log.Info($"new client connected. socket =
{session.BindSocket.Handle}");
            if (Connected != null) Connected(session);
        }
        protected override void OnDisConnected(SocketSession session)
        {
            log.Warn($"client disconnected. socket =

```

```
{session.BindSocket.Handle}");
    if (Connected != null) Disconnected(session);
}
protected override void OnMainLoop()
{
    if (clientSession.Count > 0)
    {
        var availableSocketList = clientSession.Keys;
        IList checkReadList = new List<Socket>(availableSocketList);
        IList checkWriteList = new List<Socket>(availableSocketList);
        IList checkErrorList = new List<Socket>(availableSocketList);
        Socket.Select(checkReadList, checkWriteList, checkErrorList,
(int)SharpSvrSettings.GetInstance().SelectMicroSeconds);
        foreach (var socket in checkErrorList)
        {
            RemoveClientSocket(socket as Socket);
        }
        readWriteSessionList.Clear();
        foreach (var socket in checkReadList)
        {
            SocketSession session = null;
            clientSession.TryGetValue(socket as Socket, out session);
            if (session != null)
            {
                if (session.IsError)
                {
                    RemoveClientSocket(session.BindSocket);
                    continue;
                }
                session.CanRead = true;
                readWriteSessionList.Add(session);
            }
        }
        foreach (var socket in checkWriteList)
        {
            SocketSession session = null;
            clientSession.TryGetValue(socket as Socket, out session);
            if (session != null)
            {
                if (session.IsError)
                {
                    RemoveClientSocket(session.BindSocket);
                    continue;
                }
                session.CanWrite = true;
                readWriteSessionList.Add(session);
            }
        }
        foreach (var session in readWriteSessionList)
        {
```

```

        if (session.IsConnected)
        {
            ThreadPool.QueueUserWorkItem(WaitCallback, session);
        }
        else RemoveClientSocket(session.BindSocket);
    }
}

protected override void OnRecvData(SocketSession session, uint len)
{
    log.Debug($"on recv data: socket={session.BindSocket.Handle}, len={len}");
    if (RecvData != null) RecvData(session);
}

protected override void OnSendData(SocketSession session, uint len)
{
    log.Debug($"on send data: socket={session.BindSocket.Handle}, len={len}");
    if (SendData != null) SendData(session);
}

protected override void OnShutDown()
{
    int tryCount = 0;
    serverSession.Close();
    var keys = clientSession.Keys;
    foreach (var socket in keys)
    {
        RemoveClientSocket(socket);
    }
    while (++tryCount <= SharpSvrSettings.GetInstance().ShutDownTryCount)
    {
        Thread.Sleep(1000); //, ...
        int maxWorkerThreads, workerThreads;
        int portThreads;
        ThreadPool.GetMaxThreads(out maxWorkerThreads, out portThreads);
        ThreadPool.GetAvailableThreads(out workerThreads, out portThreads);
        if (maxWorkerThreads - workerThreads == 0)
        {
            log.Warn(" server shutdown success!");
            break;
        }
    }
}

private void FinishAccept(IAsyncResult ia)
{
    Socket clientSocket = serverSession.BindSocket.EndAccept(ia);
    AddClientSocket(clientSocket);
    serverSession.BindSocket.BeginAccept(FinishAccept, null);
}

protected override void OnStartUp(string ip, short port)

```

```

        {
            var socket = new Socket(AddressFamily.InterNetwork, SocketType.Stream,
ProtocolType.Tcp);
            IPAddress ipAddress = null;
            ipAddress = Dns.GetHostEntry(ip).AddressList.FirstOrDefault(p =>
p.AddressFamily == AddressFamily.InterNetwork) ??
Dns.GetHostEntry(ip).AddressList[0];
            IPEndPoint endPoint = new IPEndPoint(ipAddress, port);
            socket.Bind(endPoint);
            socket.Listen((int)SharpSvrSettings.GetInstance().ListenBacklog);
            serverSession.WithSocket(socket);
            serverSession.BindSocket.BeginAccept(FinishAccept, null);
        }
    }
}
using System.Net;
using System.Net.Sockets;
using sharpsvr.common;
using sharpsvr.core;
namespace sharpsvr.net
{
    public class UdpServer : SocketServer
    {
        private static log4net.ILog log = Logger.GetInstance().GetLog();
        private IPEndPoint EndPoint { get; set; }
        public UdpServer() : base()
        {
        }
        protected override void BeforeRecvData(SocketSession session, byte[] bytes,
uint from, uint to)
        {
            log.Debug($"receive data: {System.Text.Encoding.UTF8.GetString(bytes,
(int)from, (int)(from + to)).TrimEnd()}");
        }
        protected override void BeforeSendData(SocketSession session, byte[] bytes,
uint from, uint to)
        {
            log.Debug($"send data: {System.Text.Encoding.UTF8.GetString(bytes,
(int)from, (int)(from + to))}");
        }
        protected override void OnConnected(SocketSession session)
        {
            throw new System.NotImplementedException();
        }
        protected override void OnDisconnected(SocketSession session)
        {
            throw new System.NotImplementedException();
        }
        protected override void OnMainLoop()
        {

```

```

        try
        {
            EndPoint endPoint = new IPEndPoint(IPAddress.Any, 0);
            var len = serverSession.DoRecvFrom(endPoint);
            if (len > 0)
            {
                log.Info($"udp has received {len} data.");
            }
        }
        catch (SocketException ex)
        {
            if (ex.Message.Contains("timed out")) return;
            else
            {
                log.Error("main loop error!", ex);
                throw ex;
            }
        }
    }

    protected override void OnRecvData(SocketSession session, uint len)
    {
        throw new System.NotImplementedException();
    }

    protected override void OnSendData(SocketSession session, uint len)
    {
        throw new System.NotImplementedException();
    }

    protected override void OnShutDown()
    {
        serverSession.Close();
    }

    protected override void OnStartUp(string ip, short port)
    {
        EndPoint = new IPEndPoint(IPAddress.Parse(ip), port);
        Socket server = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
        //UdpClient client = new UdpClient(EndPoint);
        server.Bind(EndPoint);
        server.ReceiveTimeout
=
(int)SharpSvrSettings.GetInstance().UdpRecvTimeOut;
        serverSession.WithSocket(server);
        serverSession.OnSend += (ref byte[] bytes, ref uint from, ref uint to)
=>
        {
            BeforeSendData(serverSession, bytes, from, to);
        };
        serverSession.OnRecv += (ref byte[] bytes, ref uint from, ref uint to)
=>
        {
            BeforeRecvData(serverSession, bytes, from, to);
        };
    }

```

```

        };
    }
}
namespace sharpsvr.protocol
{
    public interface ISerializable
    {
        byte[] Serialize(object obj);

        T Deserialize<T>(byte[] bytes, int from=0, int to=0) where T : class;
    }
}
using System.IO;
using System.Runtime.Serialization.Formatters.Binary;
using System.Threading;
using sharpsvr.attributes;
using sharpsvr.common;
namespace sharpsvr.protocol.serializer
{
    [SharpSerializer(Name="binary")]
    public class BinarySerializer : Singleton<BinarySerializer>, ISerializable
    {
        ThreadLocal<BinaryFormatter> formatter = new
        ThreadLocal<BinaryFormatter>();
        ThreadLocal<MemoryStream> stream = new ThreadLocal<MemoryStream>();
        private BinaryFormatter GetFormatter()
        {
            return formatter.IsValueCreated ? formatter.Value : formatter.Value =
            new BinaryFormatter();
        }
        private MemoryStream GetStream()
        {
            return stream.IsValueCreated ? stream.Value : stream.Value = new
            MemoryStream();
        }
        public T Deserialize<T>(byte[] bytes, int from = 0, int to = 0) where T :
        class
        {
            if (bytes == null || bytes.Length <= 0) return null;
            if (to <= 0) to = bytes.Length;
            if (to <= from) return null;
            var stream = GetStream();
            stream.SetLength(0);
            stream.Seek(0, SeekOrigin.Begin);
            stream.Write(bytes, from, to - from);
            stream.Seek(0, SeekOrigin.Begin);
            return GetFormatter().Deserialize(stream) as T;
        }
        public byte[] Serialize(object obj)
    }
}

```

```

        {
            var stream = GetStream();
            stream.Seek(0, SeekOrigin.Begin);
            stream.SetLength(0);
            GetFormatter().Serialize(stream, obj);
            return stream.ToArray();
        }
    }
}

using System.IO;
using System.Runtime.Serialization.Formatters.Binary;
using System.Threading;
using sharpsvr.common;
using Newtonsoft.Json;
using sharpsvr.attributes;
namespace sharpsvr.protocol.serializer
{
    [SharpSerializer(Name="json")]
    public class JsonSerializer : Singleton<JsonSerializer>, ISerializable
    {
        public T Deserialize<T>(byte[] bytes, int from = 0, int to = 0) where T :
class
    {
        if (bytes == null || bytes.Length <= 0) return null;
        if (to <= 0) to = bytes.Length;
        if (to <= from) return null;
        return
JsonConvert.DeserializeObject(System.Text.Encoding.UTF8.GetString(bytes, from,
to - from), typeof(T)) as T;
    }
    public byte[] Serialize(object obj)
    {
        return
System.Text.Encoding.UTF8.GetBytes(JsonConvert.SerializeObject(obj));
    }
}
}

using System;
using sharpsvr.attributes;
using sharpsvr.common;
using sharpsvr.protocol.serializer;
namespace sharpsvr.protocol
{
    public class SerializerManager : Singleton<SerializerManager>
    {
        private volatile
System.Collections.Concurrent.ConcurrentDictionary<string, ISerializable>
serializers = new
System.Collections.Concurrent.ConcurrentDictionary<string, ISerializable>();
        public ISerializable GetSerializer(string name)

```



```

        {
            ISerializable serializer = null;
            if (serializers.ContainsKey(name))
            {
                serializers.TryGetValue(name, out serializer);
                return serializer;
            }
            var assemblies = AppDomain.CurrentDomain.GetAssemblies();
            foreach (var assembly in assemblies)
            {
                foreach (var type in assembly.GetTypes())
                {
                    var attributes = type.GetCustomAttributes(typeof(SharpSerializerAttribute), false);
                    if (attributes != null && attributes.Length > 0 &&
                        name.ToLower().Equals((attributes[0] as SharpSerializerAttribute).Name.ToLower()))
                    {
                        serializer = Activator.CreateInstance(type) as ISerializable;
                        serializers[name] = serializer;
                        break;
                    }
                }
            }
            if (serializer == null) serializer = JsonSerializer.GetInstance();
            return serializer;
        }
    }
}

using System;
namespace sharpsvr.protocol
{
    [Serializable]
    public class SharpPacket
    {
        public string Encoder { get; set; }
        public long Random { get; set; }
        public string ServiceUniqueName { get; set; }
        public string Version { get; set; } = ("1.0.0");
        public string MethodUniqueName { get; set; }
        public object[] Arguments { get; set; }
        public object Result { get; set; }
        public override string ToString()
        {
            return $"SharpPacket (ServiceUniqueName={ServiceUniqueName}, Version={Version}, MethodUniqueName={MethodUniqueName}, Result={Result})";
        }
    }
}

```

```

using System;
using System.Collections.Generic;
using System.Reflection;
using log4net;
using sharpsvr.common;
using sharpsvr.core;
using sharpsvr.net;
using sharpsvr.util;
namespace sharpsvr.protocol
{
    public class SharpProtocol : Singleton<SharpProtocol>
    {
        public const string SVR0 = "svr0";
        public const string SVR = SVR0;
        public const int HEADER_LEN = 20;
        private static ILog log = Logger.GetInstance().GetLog();
        private Random random = new Random();
        public static string GetVersion(Type type)
        {
            var attribute = AttributeUtils.GetRemoteServiceAttribute(type);
            return attribute == null ? "1.0.0" : attribute.Version;
        }
        public static byte[] GetHeader(byte[] content)
        {
            uint len = (uint)content.Length;
            byte[] header = new byte[HEADER_LEN];
            var index = 0;
            string headerStr = SVR + SharpSvrSettings.GetInstance().EncoderName;
            header[0] = (byte)headerStr.Length;
            for (; index < headerStr.Length; ++index)
            {
                header[index + 1] = (byte)headerStr[index];
            }
            header[HEADER_LEN - 4] = (byte)(len & 0xff);
            header[HEADER_LEN - 3] = (byte)((len >> 8) & 0xff);
            return header;
        }
        public byte[] Encode(MethodInfo methodInfo, object[] args)
        {
            return Encode(new SharpPacket
            {
                Version = GetVersion(methodInfo.DeclaringType),
                ServiceUniqueName =
                    TypeUtils.GetTypeUniqueName(methodInfo.DeclaringType),
                MethodUniqueName = TypeUtils.GetMethodUniqueName(methodInfo),
                Arguments = args,
                Random = random.Next()
            }, false);
        }
        public byte[] Encode(SharpPacket packet, bool isServerSide = true)
    }
}

```

```

    {
        if (packet.Random == 0) packet.Random = random.Next();
        var serializer =
        SerializerManager.GetInstance().GetSerializer(SharpSvrSettings.GetInstance().EncoderName);
        if (isServerSide)
        {
            packet.MethodUniqueName = packet.ServiceUniqueName =
            packet.Version = null;
            packet.Arguments = null;
        }
        return serializer.Serialize(packet);
    }
    public SharpPacket TryDecodePacket(SocketSession socketSession)
    {
        if (socketSession.RecvBuffer.Used < HEADER_LEN) return null;
        byte headerStrLen = socketSession.RecvBuffer[0];
        for (int i = 0; i < SVR.Length; ++i)
        {
            if (socketSession.RecvBuffer[i + 1] != SVR[i])
            {
                log.Error("recv buffer error! protocol header check failed!");
                socketSession.Close();
                return null;
            }
        }
        socketSession.RecvBuffer.PeekBytes(ref header, HEADER_LEN);
        string encoder = System.Text.Encoding.ASCII.GetString(header,
        SVR.Length + 1, headerStrLen - SVR.Length);
        var fullPacketSize = (socketSession.RecvBuffer[HEADER_LEN - 4]) +
        (socketSession.RecvBuffer[HEADER_LEN - 3] << 8) +
        (socketSession.RecvBuffer[HEADER_LEN - 2] << 16) +
        (socketSession.RecvBuffer[HEADER_LEN - 1] << 24) + HEADER_LEN;
        if (fullPacketSize <= socketSession.RecvBuffer.Used)
        {
            var buffer =
            ByteArrayFactory.GetInstance().allocate((uint) (fullPacketSize));
            socketSession.RecvBuffer.ReadBytes(ref buffer,
            (uint) fullPacketSize);
            var serializer =
            SerializerManager.GetInstance().GetSerializer(encoder);
            var obj = serializer.Deserialize<SharpPacket>(buffer, HEADER_LEN,
            fullPacketSize);
            (obj as SharpPacket).Encoder = encoder;
            log.Debug($"recv packet: {obj}, client
            session: {socketSession.BindSocket.Handle}");
            return obj as SharpPacket;
        }
        return null;
    }
}

```

```

    }
}
using System;
using System.Reflection;
using sharpsvr.common;
namespace sharpsvr.proxy
{
    public abstract class Interceptor
    {
        private log4net.ILog log = Logger.GetInstance().GetLog();
        public virtual object Invoke(MethodInfo methodInfo, object[] args)
        {
            log.Info($"invoke called method={(methodInfo == null ? "null" :
methodInfo.Name)}, args.len={args==null ? 0 : args.Length}");
            if(args != null) {
                for(int i = 0; i < args.Length; ++i) {
                    log.Info($"arg[{i}]={args[i]}");
                }
            }
            return methodInfo.ReturnType.IsValueType ?
Activator.CreateInstance(methodInfo.ReturnType) : null;
        }
    }
}
using System;
using System.Threading.Tasks;
using commontest.src.test;
using sharpsvr.net;
namespace svrtest
{
    class Program
    {
        static void Main(string[] args)
        {
            var svr = new sharpsvr.net.SharpServer();
            ICalc cal = new Calc();
            Action<IServer> action = async (IServer server)=>{
                svr.WithService(cal);
                svr.StartUp();
                await Task.Run(()=> server.MainLoop());
            };
            action(svr);
            Console.ReadKey();
            svr.ShutDown();
        }
    }
}

```